

Lab 2 Worksheet

Exercise 1:

At the moment, in `geometric_inv_kin.py`, `geometric_inv_kin()` returns only `allPhi`. Edit `geometric_inv_kin()` so that we can pass in an array of joint positions as an argument (`phi_prev`), then the closest valid solution is outputted as `bestPhi`.

Description	Position (x, y, z)	wrist	phi_prev	allPhi	bestPhi
Normal	[0.3, 0.4, 0.35]	50	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ 0.87 & 0.87 & 0.87 & 0.87 \end{pmatrix}$	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	200	[0, 0, 0, 0]	$\begin{pmatrix} -2.21 & -2.21 & 0.93 & 0.93 \\ -2.14 & -0.49 & 0.2 & 1.86 \\ 0.11 & -2.97 & 0.11 & -2.97 \\ -2.79 & -2.79 & -2.79 & -2.79 \end{pmatrix}$	Fail
Negative Position	[-0.2, -0.2, 0.6]	-150	[0, 0, 0, 0]	$\begin{pmatrix} 0.79 & 0.79 & -2.36 & -2.36 \\ -1.53 & 0.14 & -0.42 & 1.24 \\ 0.12 & -2.98 & 0.12 & -2.98 \\ -2.62 & -2.62 & -2.62 & -2.62 \end{pmatrix}$	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	$\begin{pmatrix} -3.14 & 0. & -3.14 & 0. \\ -0.82 & 0.53 & -0.82 & 0.53 \\ -0.17 & -2.69 & -0.17 & -2.69 \\ 0. & 0. & 0. & 0. \end{pmatrix}$	Fail

Exercise 2: In `numerical_inv_kin.py`, Implement a numerical solver for the QArm using the provided `differential_kinematics()` function to get the inverse Jacobian. Use the same test cases as before.

Description	Position (x, y, z)	wrist	phi_prev	bestPhi
Normal	[0.3, 0.4, 0.35]	50	[0, 0, 0, 0]	$\begin{pmatrix} 0.93 \\ 0.2 \\ 0.11 \\ 0.87 \end{pmatrix}$
Invalid wrist	[0.3, 0.4, 0.35]	200	[0, 0, 0, 0]	Fail
Negative Position	[-0.2, -0.2, 0.6]	-150	[0, 0, 0, 0]	$\begin{pmatrix} -2.36 \\ -0.42 \\ 0.12 \\ -2.62 \end{pmatrix}$
Out of workspace	[0.75, 0.3, 0.5]	0	[0, 0, 0, 0]	Fail
Singularity	[0, 0, 0.75]	0	[0, 0, 0, 0]	Fail

Note: since the numerical solver stops at a threshold, your values will be slightly different than the exact values provided by the geometric approach.

Exercise 3: Compare the two solvers, what are their pros and cons?

A few key ideas to think about, but add your own ideas as well:

- Can you think of reasons you would choose to use one solver or the other?
- How consistent are the solvers? (in terms of final position and time taken)

Exercise 4: Can you think of modifications that could improve each method?

Exercise 5: Create an RRT algorithm in the file `obstacle_avoidance_RRT.py`.

Note: be careful with step sizes, make it too large and you may “skip” through an obstacle

Test and demonstrate your program in all obstacle setups.

To help you manage a tree, the `RRT_Tree_Manager` has been implemented. It will track nodes and keep a parallel list to track parents. It also has the following functions:

Methods	Description	Inputs	Outputs
<code>add_node(q_new, parent_index)</code>	Adds a node to the list, and records its parent node	<code>q_new, parent_index</code>	-
<code>get_nearest_neighbor(q_rand)</code>	Finds the node closest to <code>q_rand</code>	<code>q_rand</code>	<code>node, node_idx</code>
<code>get_size()</code>	Returns the number of nodes in the tree	-	<code>size</code>

The `RRT_helpers` class is implemented with the following functions to help with implementation:

Methods	Description	Inputs	Outputs
<code>is_configuration_colliding(phi)</code>	checks if the <code>phi</code> is valid and avoids collision with obstacles	<code>joint_angles</code>	True if invalid position, False otherwise

Exercise 6: In `obstacle_avoidance_potential.py` implement obstacle avoidance using potential fields.

Test and demonstrate your program with all obstacle setups.

The `Potential_Fields_helper` class is implemented with the following functions to help with implementation:

Methods	Description	Inputs	Outputs
<code>get_all_obstacle_distances(self, p4)</code>	returns a list of dictionaries containing distances and directions (EE to obstacle)	<code>p4</code> (EE position in {0} system)	<code>obstacle_data</code>

Tips:

- use `J = QArm_Interface.Jacobian(phi)` to get the 4×4 analytic Jacobian.
 - Don't forget to append a zero to the force vector before multiplying (for the wrist that we are ignoring)
- Use `QArm_Interface.write_to_arm(phi)` to move the arm to the joint positions specified by `phi`

Exercise 7: Now that you have experienced both methods, list some advantages and disadvantages of each method has.

A few key ideas to think about, but list more if you think of anything:

- How “fast” are they? (Assuming similar step sizes, but including processing times)
- Can the manipulator get “stuck”? Why? (Assuming it is possible to reach the destination)
- How would they handle a environment with (predictably) moving obstacles?
- How would they scale to more complex obstacles and environments?
- How would they scale to more complex manipulators?

Exercise 8: What are some improvements you can think of to make to each method?

A few key ideas to think about, but don’t stop there:

- How could you adapt them to work in unpredictable environments? (Assuming you had some sort of live map of the environment)
- Could you improve the way each method moves (or is unable to move)?